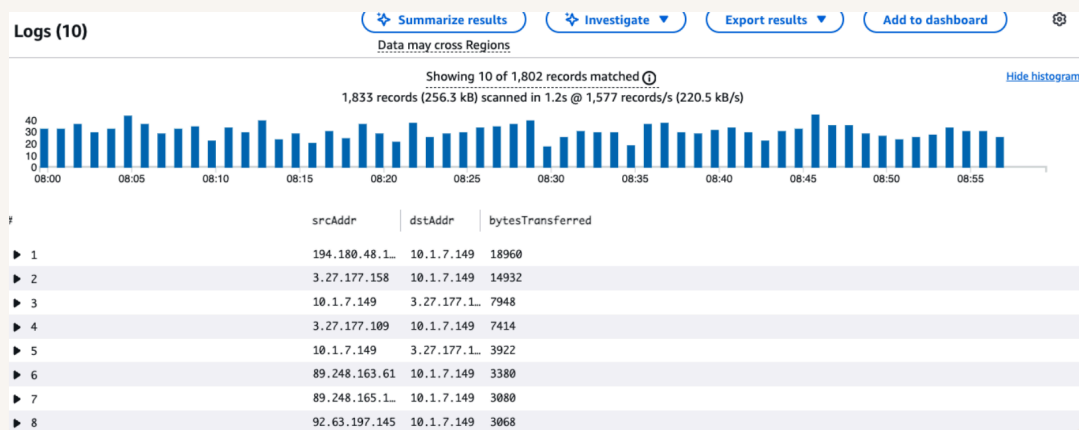




VPC Monitoring with Flow Logs



Pelin Ece Burgun





Introducing Today's Project!

What is Amazon VPC?

Amazon VPC is an isolated, user-defined virtual network within AWS. It's useful because it gives full control over IP addressing, subnet structure, external connectivity (internet gateway, NAT gateway, peering, endpoints), and security boundaries (security groups, NACLs) — letting you design network architecture that matches your actual security and access requirements instead of relying on defaults.

How I used Amazon VPC in this project

I used two VPCs (NextWork-1 and NextWork-2) with non-overlapping CIDR blocks, connected them via a peering connection, and set up VPC Flow Logs on one VPC's subnet to capture and analyze traffic between EC2 instances in real time.

One thing I didn't expect in this project was...

I didn't expect that accepting a peering connection wouldn't automatically enable communication — you still need to manually add routes in both VPCs' route tables. The peering connection is just the physical link; routing is a separate, explicit step. That's a gap worth remembering for real deployments — it's an easy thing to forget and quietly leaves resources unreachable.

This project took me...

Roughly 60 minutes, in line with the project's estimated time — most of it went into the IAM policy/role setup and troubleshooting the failed private IP ping before finding the missing peering route.



In the first part of my project...

Step 1 - Set up VPCs

In this step, I will create two VPCs using the VPC wizard, each with one public subnet, no NAT gateway, and no VPC endpoints, because I need two separate isolated networks to later connect via peering — the whole point of this project is demonstrating cross-VPC traffic monitoring, which requires at least two VPCs to exist first.

Step 2 - Launch EC2 instances

In this step, I will launch one EC2 instance in each VPC, because I need actual endpoints to generate traffic between — without instances, there's nothing for the peering connection or Flow Logs to actually demonstrate.

Step 3 - Set up Logs

In this step, I will create a CloudWatch log group and a VPC Flow Log, because Flow Logs needs a destination to send captured traffic data to — the log group is where that data will actually live and be queryable.

Step 4 - Set IAM permissions for Logs

In this step, I will create an IAM policy and role for VPC Flow Logs, because Flow Logs has no default permission to write data to CloudWatch — without this, the flow log configuration can't be completed since there's no IAM role available to select.



Multi-VPC Architecture

I created two separate VPCs: NextWork-1 with CIDR block 10.1.0.0/16 and NextWork-2 with CIDR block 10.2.0.0/16, each with 1 public subnet, 0 private subnets, no NAT gateway, and no VPC endpoints. Total: 2 subnets across 2 VPCs.

Each VPC needs a non-overlapping CIDR block so that IP addresses across the two networks don't collide. This matters specifically because of the peering connection — if VPC 1 and VPC 2 shared or overlapping ranges, routing tables couldn't unambiguously determine which VPC a given private IP belongs to, breaking the very peering setup this project depends on. It's not just a naming convenience; it's a hard technical requirement for peering to function at all.

I also launched EC2 instances in each subnet

Both security groups (NextWork-1-SG and NextWork-2-SG) allow inbound ICMP traffic from 0.0.0.0/0 — all IPv4 addresses. This is broader than good practice for a production environment (ICMP from anywhere has no real use case outside testing), but it's deliberately permissive here so the ping tests later in the project aren't blocked by security group rules, isolating the routing issue as the actual variable being tested.



Ön izleme

VPC [Ayrıntıları göster](#)

AWS sanal ağıınız

NextWork-2-vpc

Alt ağlar (1)

Bu VPC içindeki alt ağlar

eu-central-1a

 NextWork-2-subnet-public1-eu-

Yönlendirme tabloları (1)

Ağ trafiğini kaynaklara yönlendir

NextWork-2-rtb-public



Logs

Logs are records of system activity — who did what, when, and any errors that occurred. They're the primary tool for troubleshooting and monitoring system behavior over time.

A log group is a container in CloudWatch for organizing related logs — in this case, all flow log records from a given VPC/subnet get sent there. Retention (how long logs are kept) and log class (Standard vs. Infrequent Access) are set at the log group level, not per individual log entry.

fl-0072970e213a52de8 / NextWorkVPCFlowLogsPolicy

Ayrıntılar

Akış Günlüğü Kimliği

fl-0072970e213a52de8

Hedef Türü

cloud-watch-logs

Trafik Türü

Tümü

Ad

NextWorkVPCFlowLogsPolicy

Hedef Adı

[NextWorkVPCFlowLogsGroup](#)

Maksimum Toplama Aralığı

1 dakika

Durum

Aktif

IAM Rolü

[arn:aws:iam::825765392980:role/NextWorkVPCFlowLogsRole](#)

Günlük Biçimi

Varsayılan

Oluşturulma Zamanı

4 Temmuz 2026 Cumartesi 13:47:20
GMT+3

Çapraz Hesap IAM Rolü

-

Tag field specifications

-



IAM Policy and Roles

I created an IAM policy because it defines the specific CloudWatch Logs actions (CreateLogGroup, CreateLogStream, PutLogEvents, DescribeLogGroups, DescribeLogStreams) that Flow Logs needs. A policy alone grants nothing to a service — it has to be attached to a role, which is the actual entity AWS services assume.

I created an IAM role because policies can't be attached directly to a service — they have to be bundled into a role first. I used a custom trust policy scoped specifically to `vpc-flow-logs.amazonaws.com`, so only VPC Flow Logs (not any other service or user) can assume this role.

A custom trust policy defines who is allowed to assume an IAM role — it's separate from the permissions policy, which defines what that entity can do once it has the role. In this case, I scoped the trust policy to `vpc-flow-logs.amazonaws.com` specifically, so only VPC Flow Logs can use this role — not any other service, even if someone accidentally tried to attach it elsewhere.



Özel güven politikası

Başkalarının bu hesapta eylemler gerçekleştirmesini sağlamak için özel bir güven politikası oluşturun.

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Sid": "Statement1",  
6       "Effect": "Allow",  
7       "Principal": {"Service": "vpc-flow-logs.amazonaws.com"},  
8     },  
9     {  
10      "Action": "sts:AssumeRole"  
11    }  
12  ]  
13 }
```



In the second part of my project...

Step 5 - Ping testing and troubleshooting

In this step, I will generate test traffic between Instance 1 and Instance 2 using ping, because this validates the VPC peering setup (or reveals it's incomplete) and simultaneously gives Flow Logs real traffic to capture for the analysis in Step 7.

Step 6 - Set up a peering connection

In this step, I will create a peering connection between VPC 1 and VPC 2 and update both route tables, because a peering connection alone doesn't route traffic — each VPC needs an explicit route pointing the other's CIDR block to the peering connection, or private IP traffic has no path between them.

Step 7 - Analyze flow logs

In this step, I will review the flow logs generated by VPC Flow Logs and use CloudWatch Logs Insights to query them, because raw logs alone are hard to interpret at scale — running a query (like "Top 10 byte transfers by source and destination IP") turns individual log entries into an actionable view of which traffic patterns dominate the network.



Connectivity troubleshooting

My first ping test between the EC2 instances got no replies, which means there was no working path between the two VPCs at the private IP level — despite both security groups allowing ICMP. The problem was routing, not security configuration.

```

~\_#_##### Amazon Linux 2023
~~~\_#_#####
~~~\_###|
~~~\_#/ https://aws.amazon.com/linux/amazon-linux-2023
~~~~V~'-'>
~~~~
~~~~~
~~~~~.-.-.-
~~~~~/././././m/'-'
[ec2-user@ip-10-1-4-114 ~]$ ping 10.2.5.67
PING 10.2.5.67 (10.2.5.67) 56(84) bytes of data.
```

I could receive ping replies using the other instance's public IP address, which means Instance 2 was correctly configured to accept ICMP traffic and was reachable — but only via the public internet through the internet gateway, not through a direct private connection. This confirmed the issue was specifically the missing peering route, not a broader connectivity or security group problem.



Connectivity troubleshooting

Looking at VPC 1's route table, the ping test failed because there was no route for VPC 2's CIDR block (10.2.0.0/16) — only the default 0.0.0.0/0 route to the internet gateway existed. Without a peering connection and matching routes in place, private IP traffic between the two VPCs has nowhere to go.

To solve this, I set up a peering connection between my VPCs

I updated both VPCs' route tables because accepting a peering connection doesn't automatically create routing paths. Each VPC needed an explicit route pointing the other VPC's CIDR block (10.2.0.0/16 and 10.1.0.0/16 respectively) to the peering connection as the target.

rtb-0b4bd82a287d6016b / NextWork-1-rtb-public

Ayrıntılar

Yönlendirmeler

Alt ağ ilişkilendirmeleri

Uç ilişkilendirmeleri

Yönlendirme yayılımı

Etiketler

Yönlendirmeler (3)

İkisi de

Yönlendirmeleri düzenle

Yönlendirmeleri filtrele

<1>

Hedef	Hedef	Durum	Yayıldı	Route Origin
0.0.0.0/0	igw-09d85ae667cd3380d	✓ Aktif	Hayır	Create Route
10.1.0.0/16	local	✓ Aktif	Hayır	Create Route Table
10.2.0.0/16	pcx-00f3776835bb0bbd5	✓ Aktif	Hayır	Create Route



Connectivity troubleshooting

Getting ping replies from Instance 2's private IP confirmed the peering connection and route table updates worked — traffic between the two instances now flows directly through AWS's private network instead of over the public internet.

```
[ec2-user@ip-10-1-4-114 ~]$ ping 10.2.5.67
PING 10.2.5.67 (10.2.5.67) 56(84) bytes of data.
64 bytes from 10.2.5.67: icmp_seq=1 ttl=127 time=1.01 ms
64 bytes from 10.2.5.67: icmp_seq=2 ttl=127 time=0.484 ms
64 bytes from 10.2.5.67: icmp_seq=3 ttl=127 time=0.461 ms
64 bytes from 10.2.5.67: icmp_seq=4 ttl=127 time=0.470 ms
64 bytes from 10.2.5.67: icmp_seq=5 ttl=127 time=0.471 ms
64 bytes from 10.2.5.67: icmp_seq=6 ttl=127 time=0.508 ms
64 bytes from 10.2.5.67: icmp_seq=7 ttl=127 time=0.462 ms
64 bytes from 10.2.5.67: icmp_seq=8 ttl=127 time=0.459 ms
64 bytes from 10.2.5.67: icmp_seq=9 ttl=127 time=0.452 ms
64 bytes from 10.2.5.67: icmp_seq=10 ttl=127 time=0.475 ms
64 bytes from 10.2.5.67: icmp_seq=11 ttl=127 time=0.445 ms
64 bytes from 10.2.5.67: icmp_seq=12 ttl=127 time=0.457 ms
```



Analyzing flow logs

Flow logs contain: source IP, destination IP, port, protocol (TCP/UDP/ICMP), packets and bytes transferred, and whether the traffic was ACCEPT or REJECT.

This flow log shows 344 bytes of data sent from 18.237.140.165 to 10.1.5.112 over TCP on port 22, with 4 packets transferred, and the traffic was ACCEPT — meaning it was allowed through by the security group and NACL. The source IP falls in EC2 Instance Connect's IP range for the Oregon region, so this specific log is Instance Connect accessing Instance 1's SSH port, not the ping traffic between instances.

▶	Zaman damgası	Mesaj
▶	2026-07-04T11:42:07.000Z	2 825765392980 eni-059108c3df0f19d8a 89.248.171.23 10.1.4.114 46830 443 6 7 420 1783165327 1783165352 REJECT OK
▶	2026-07-04T11:42:07.000Z	2 825765392980 eni-059108c3df0f19d8a 93.174.93.12 10.1.4.114 50652 443 6 7 420 1783165327 1783165352 REJECT OK
▶	2026-07-04T11:42:07.000Z	2 825765392980 eni-059108c3df0f19d8a 147.185.132.70 10.1.4.114 50204 3260 6 1 44 1783165327 1783165352 REJECT OK
▶	2026-07-04T11:42:37.000Z	2 825765392980 eni-059108c3df0f19d8a 3.120.181.44 10.1.4.114 5097 22 6 4 344 1783165357 1783165359 ACCEPT OK
▶	2026-07-04T11:42:37.000Z	2 825765392980 eni-059108c3df0f19d8a 10.1.4.114 3.120.181.44 22 5097 6 2 176 1783165357 1783165359 ACCEPT OK
▶	2026-07-04T11:42:37.000Z	2 825765392980 eni-059108c3df0f19d8a 185.141.54.174 10.1.4.114 0 0 47 1 52 1783165357 1783165359 REJECT OK
▶	2026-07-04T11:43:06.000Z	2 825765392980 eni-059108c3df0f19d8a 20.65.193.148 10.1.4.114 60084 1080 6 1 52 1783165386 1783165406 REJECT OK
▶	2026-07-04T11:43:06.000Z	2 825765392980 eni-059108c3df0f19d8a 35.203.210.127 10.1.4.114 54628 54322 6 1 44 1783165386 1783165406 REJECT OK
▶	2026-07-04T11:43:06.000Z	2 825765392980 eni-059108c3df0f19d8a 85.217.140.11 10.1.4.114 59919 14116 6 1 52 1783165386 1783165406 REJECT OK
▶	2026-07-04T11:43:06.000Z	2 825765392980 eni-059108c3df0f19d8a 109.205.211.91 10.1.4.114 56844 31903 6 1 44 1783165386 1783165406 REJECT OK
▶	2026-07-04T11:43:06.000Z	2 825765392980 eni-059108c3df0f19d8a 147.185.132.83 10.1.4.114 53951 1935 6 1 44 1783165386 1783165406 REJECT OK

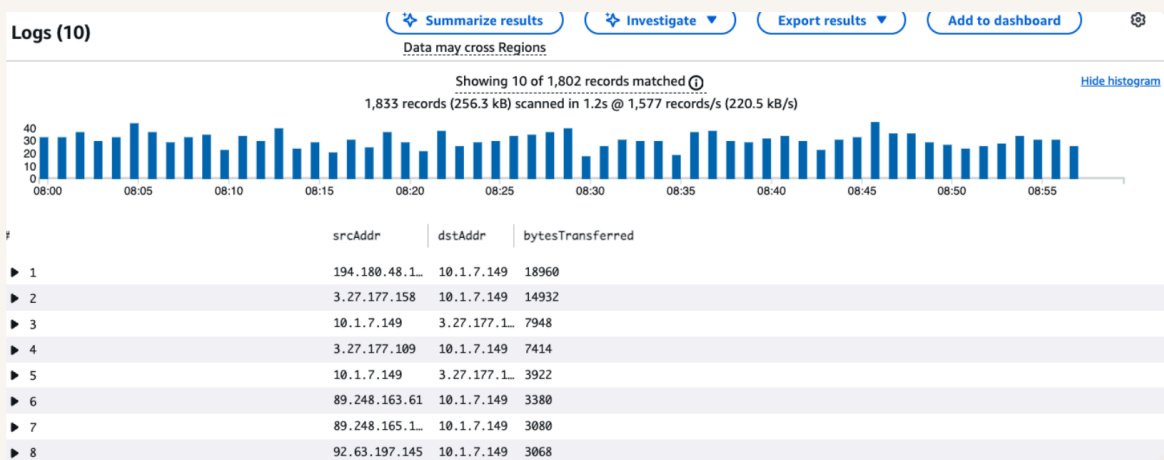
Başla dönün ^



Logs Insights

Logs Insights is a CloudWatch feature for querying log data — filtering, grouping, and aggregating (sums, averages, top-N) to identify traffic patterns, anomalies, or troubleshoot issues without manually scanning raw logs.

I ran "Top 10 byte transfers by source and destination IP addresses" against NextWorkVPCFlowLogsGroup. It aggregates flow log records by IP pair and ranks them by total bytes transferred, surfacing which source/destination pairs are moving the most data. Practically, this is the query you'd run first when investigating unexpected bandwidth usage or checking whether traffic is going where you expect — if an unfamiliar IP shows up near the top, that's your signal to dig further.





nextwork.ai

The place to learn & showcase your skills

Check out nextwork.ai for more projects

